

C.Arch Project Specifications

Team Members:

1. Rishikesh Kasireddy - ES23BTECH11033
2. Sai Soham Venkata Mashetty- ES23BTECH11036

Scope

This project aims to extend the existing single-cycle in-house simulator into a fully functional **pipelined processor simulator** that accurately models the instruction-level parallelism of a modern RISC-style CPU. The simulator will be capable of demonstrating **five distinct execution modes**, each progressively introducing real-world architectural mechanisms such as hazard detection, forwarding, and branch prediction.

The project will not only focus on correctness and functional verification but also provide **performance insights** (cycle counts, CPI, stall rates, and mispredictions) for various benchmark programs.

Key Deliverables

1. Extend the simulator from a single-cycle to a 5-stage pipelined implementation (IF, ID, EX, MEM, WB).
2. Implement and demonstrate the following pipeline configurations:
 - a. **Mode 1:** Single-cycle baseline (existing)
 - b. **Mode 2:** Pipelined (no hazard handling)
 - c. **Mode 3:** Pipelined with hazard detection (stalls)
 - d. **Mode 4:** Pipelined with hazard detection + forwarding
 - e. **Mode 5:** With static branch prediction
 - f. **Mode 6:** With dynamic 1-bit prediction
3. Provide detailed cycle-accurate simulation of pipeline behavior.
4. Develop a **Performance Dashboard** to automatically measure and compare metrics (total cycles, CPI, stall count, forwarding events, mispredictions).

Stretch Goals (If time permits)

1. **2-Bit Dynamic Branch Predictor:** Extend the branch prediction module to include a 2-bit saturating counter predictor and compare misprediction rates to the 1-bit predictor
2. **Live Pipeline Visualization:** During program execution, the simulator will highlight multiple instructions simultaneously, showing which stage each instruction is in (IF, ID, EX, MEM, WB) in real-time.

High-Level Project Plan (4 Weeks)

1. Week 1:

- a. Finalize project specification.
- b. Design the pipelined datapath architecture.
- c. Implement the 5 pipeline stages and the intervening pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB).
- d. Achieve a functional pipeline without hazard handling (Mode 2).
- e. Validate basic instruction execution for correctness and establish baseline performance metrics (cycle counts, CPI).

2. Week 2:

- a. Design and implement the Hazard Detection Unit for stalling (Mode 3).
- b. Design and implement the Forwarding Unit to resolve EX/MEM and MEM/WB hazards (Mode 4).
- c. Handle the specific case of the load-use hazard.

3. Week 3:

- a. Implement control hazard logic (pipeline flushing on mispredicted branches).
- b. Add support for static (Branch-Not-Taken) prediction (Mode 5).
- c. Implement the Branch History Table and logic for 1-bit dynamic prediction (Mode 6).
- d. Evaluate branch prediction accuracy and performance improvements compared to static prediction.

4. Week 4:

- a. Thoroughly test all modes with the benchmark programs.
- b. Implement the **Performance Dashboard / Report Generator**: automatically generate reports summarizing total cycles, CPI, stalls, mispredictions, and forwarding events for each mode.
- c. Collect performance data and analyze pipeline efficiency across configurations.
- d. Prepare the final project report, source code, and a presentation with demonstration.